

PEC 1: Búsqueda y Juegos

Presentación

Primera PEC del curso de Inteligencia Artificial

Competencias

En esta PEC se trabajarán las siguientes competencias:

Competencias de grado:

- Capacidad de analizar un problema con el nivel de abstracción adecuado a cada situación y aplicar las habilidades y conocimientos adquiridos para abordarlo y solucionarlo.

Competencias específicas:

- Saber representar las particularidades de un problema según un modelo de representación del conocimiento.
- Saber resolver problemas intratables a partir de razonamientos aproximados y heurísticos (algoritmos voraces, algoritmos genéticos, lógica difusa, redes bayesianas, redes neuronales, min-max).

Objetivos

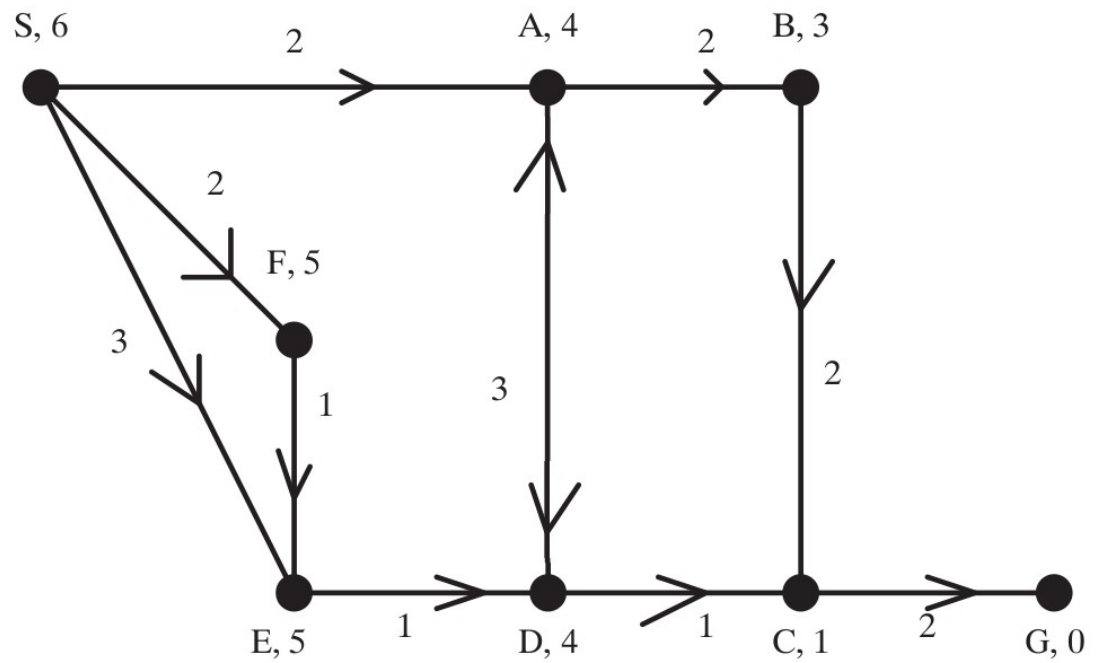
Esta PEC pretende evaluar vuestros conocimientos sobre búsquedas informadas (formalización y estrategias relacionadas) y juegos (minimax y poda alfa-beta).

Descripción de la PEC/práctica a realizar

Esta PEC tiene como objetivo explorar principalmente las búsquedas informadas y la poda α - β .

Apartado A.- Búsquedas

Sea el siguiente grafo dirigido:



donde el coste de las aristas es el indicado en las aristas y el heurístico para cada nodo es el número que figura junto al nombre del nodo (por ejemplo, en E, 5, 5 es el heurístico para el nodo E, es decir $h(E) = 5$).

En el programa Python adjunto, que recoge la infraestructura que hay descrita a los materiales para realizar búsquedas, se tienen los ejemplos de la búsqueda A* y la búsqueda uniforme ya implementados. Además, la descripción de este grafo ya está codificada en las estructuras de datos que hay que utilizar para realizar búsquedas sobre el grafo.

Preguntas:

1.- (40% del apartado A) Utilizando el programa Python adjunto, implementar aquello que haga falta para poder realizar una búsqueda ávida (es decir, la función `problema_Grafo_busqueda_Avida`) y utilizarla en este grafo. **¿Cuál es el resultado?** *Pista:* Fijarse en la implementación de la función correspondiente a la búsqueda uniforme.

El resultado es `['StoA', 'AtoB', 'BtoC', 'CtoG']`

y la implementación de la búsqueda ávida es:

```
def problema_Grafo_busqueda_Avida():
    tl_ops = tl_operadores_grafo()
    def aux_func(info_nodo_padre, estado, operador):
        return [0, heuristico(estado)]
    estado_inicial = 'S'
    check_estado_final = lambda estado: estado == 'G'
    return [tl_ops, aux_func, estado_inicial, check_estado_final,
            lambda estado: [0, heuristico(estado)]]
```

2.- (40% del apartado A) Realice ahora una búsqueda A* sobre este grafo. **Hágase paso a paso, mostrando el contenido de las listas de nodos abiertos y cerrados.** Puede comprobar si el resultado es correcto utilizando el código Python (incluso puede modificar el programa para que muestre el resultado paso a paso, si no desea hacerlo a mano). **¿Qué resultado se obtiene?**

Hemos escogido modificar el programa cambiando muy poco la función **búsqueda**, añadiendo:

```
print(mapcar(lambda nodo: [nodo[1], nodo[5]], car(arbol)))
print(mapcar(lambda nodo: [nodo[1], nodo[5]], cadr(arbol)))
print()
```

justo después de la asignación a **nuevo_arbol**

El resultado es:

```
[['S', 6]]
[]
```

```
[['A', 6], ['F', 7], ['E', 8]]
[['S', 6]]
```

```
[['F', 7], ['B', 7], ['E', 8], ['D', 9]]
[['A', 6], ['S', 6]]
```

```
[['B', 7], ['E', 8], ['E', 8], ['D', 9]]
[['F', 7], ['A', 6], ['S', 6]]
```

```
[['C', 7], ['E', 8], ['E', 8], ['D', 9]]
[['B', 7], ['F', 7], ['A', 6], ['S', 6]]
```

['E', 8], ['E', 8], ['G', 8], ['D', 9]
['C', 7], ['B', 7], ['F', 7], ['A', 6], ['S', 6]]

['G', 8], ['E', 8], ['D', 8], ['D', 9]
['E', 8], ['C', 7], ['B', 7], ['F', 7], ['A', 6], ['S', 6]]

['StoA', 'AtoB', 'BtoC', 'CtoG']

con coste 8

3.- (20% del apartado A) ¿El resultado obtenido en la pregunta 2 es óptimo? ¿Es admisible el heurístico? Si no lo es, utilice un heurístico admisible para obtener el resultado óptimo. No hay que hacerlo paso a paso. Modifique el heurístico del programa para que proporcione la solución óptima automáticamente. Comparar el resultado de aplicar la búsqueda uniforme, la búsqueda ávida y la búsqueda A* (utilizando el programa, claro está).

Una inspección visual encuentra fácilmente el camino SEDCG de coste 7, por lo tanto la solución encontrada **NO** es óptima. Esto significa que el heurístico **NO** es admisible (si lo fuese, no podría ser que el algoritmo no encontrase el óptimo, nos lo dice un teorema).

No cuesta demasiado encontrar que el heurístico $h(D)$ i $h(E)$ son los responsables. Si hacemos $h(D)=2$ y $h(E)=2$, el resultado encontrado por la búsqueda A* es **['StoF', 'FtoE', 'EtoD', 'DtoC', 'CtoG'] de cost 7.**

Búsqueda Uniforme: **['StoF', 'FtoE', 'EtoD', 'DtoC', 'CtoG'] de cost 7**

Búsqueda Ávida: **['StoE', 'EtoD', 'DtoC', 'CtoG'] de cost 7**

Apartado B.- Las Bolas y el Dado

Tenemos sobre una mesa una colección de N bolas y un dado. Este juego lo juegan dos jugadores. En cada turno un jugador mueve el dado 90° a cualquiera de las cuatro posiciones posibles (tenga en cuenta que no lo tira, sino que lo mueve a una posición elegida por él, es decir, que este NO es un juego de azar) y saca de la tabla el número indicado de bolas. Así, si al inicio de un turno el dado tenía un 1 o un 6 mirando hacia arriba, el jugador podrá elegir sacar 2, 3, 4 o 5 bolas, ya que podrá mover el dado a cualquiera de estos números mirando hacia arriba. Pierde el jugador que se queda sin poder hacer ningún movimiento.

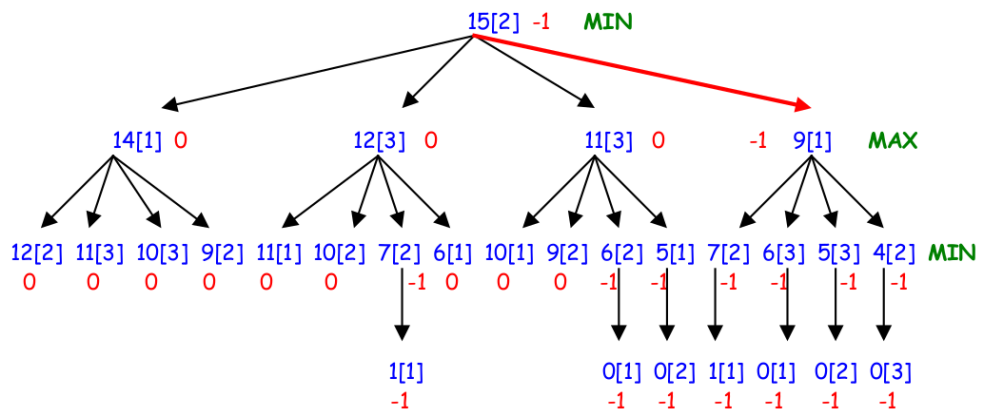
Al comenzar una jugada, las posibilidades de mover el dado vienen determinadas por el número que hay mirando hacia arriba, y son las mismas tanto si sale un número como el que está en la cara opuesta: Así, da igual que haya un 1 o un 6, un 2 o un 5 o un 3 o un 4. Denotaremos el estado del juego con un número entero N (número de bolas sobre la mesa) y un número que indicará el eje del dado que mira hacia arriba entre $[]$ (hay tres ejes posibles: 1-6, 2-5, 3-4 que denotaremos $[1]$, $[2]$, $[3]$): $N[\text{eje}]$.

Supondremos dos jugadores, A y B y habrá que utilizar la siguiente función como función de utilidad: +1 si es un estado donde ha ganado A, -1 si es un estado donde ha ganado B (0 si no ha pasado ninguna de las dos cosas, si ampliamos la función de utilidad a una función heurística para evaluar cualquier estado).

Preguntas:

1.- (30% del apartado B) Con estado inicial 15[2] y turno para el jugador B (jugador min), construir el árbol minimax hasta profundidad 3 (jugada de B, respuesta de A y jugada de B) ¿Qué jugada inicial elegiría B? ¿Tiene la victoria asegurada con algún movimiento? Discuta las posibilidades de B.

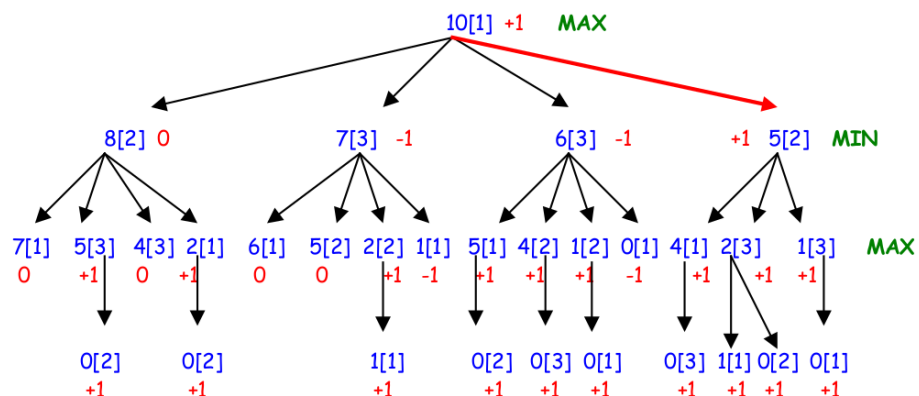
Sólo pondremos, en el último nivel del árbol, aquellos nodos que llevan a un resultado diferente de cero para la función de utilidad.



Y el jugador B elegirá la jugada indicada en rojo (poner el 6 hacia arriba).

2.- (30% del apartado B) Hacer lo mismo con estado inicial 10[1] con turno para el jugador A (jugador max), construir el árbol minimax hasta profundidad 3 (jugada de A, respuesta de B, de nuevo jugada de A). ¿Qué jugada inicial elegiría A? ¿Tiene la victoria asegurada con algún movimiento?

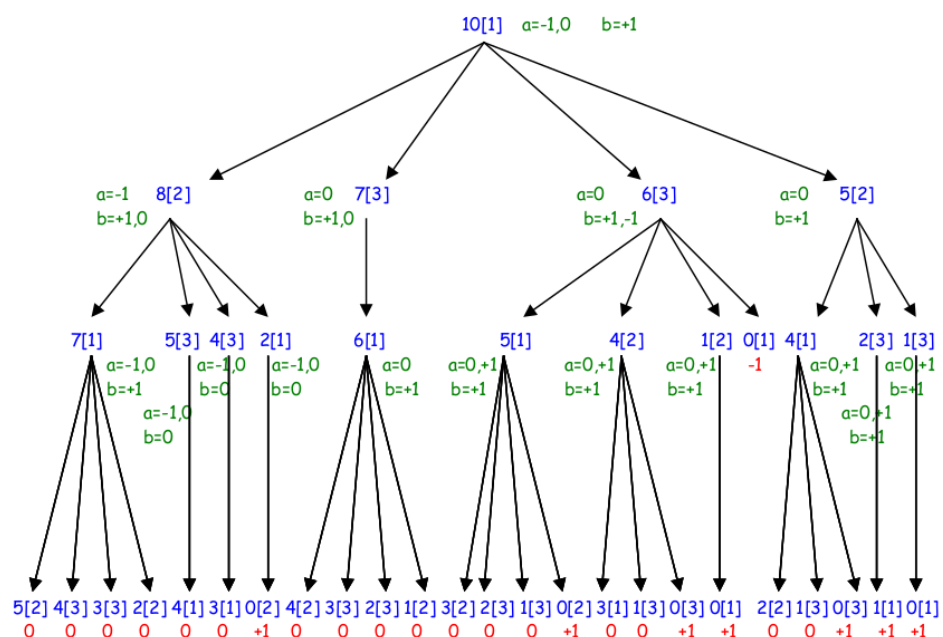
Haremos lo mismo que en el ejercicio anterior: Sólo pondremos, en el último nivel del árbol, aquellos nodos que llevan a un resultado diferente de cero para la función de utilidad.



Y el jugador A elegirá la jugada indicada en rojo (poner el 5 hacia arriba).

3.- (40% del apartado B) Aplicar la poda α - β al árbol que resulta de la pregunta 2 y comentar el resultado.

El resultado de la poda dependerá mucho del orden en que se aplican las posibles jugadas (nuestros operadores) para generar diferentes hijos. Aquí os pondré una posible solución, donde genero los hijos en orden, es decir, si estoy en el estado 10 [1] y puedo jugar a poner el dado en 2,3,4 y 5, generaré los hijos de 10 [1] en este mismo orden creciente y de izquierda a derecha.



Evidentemente, a es α y b es β . Cuando hay una expresión de la forma $a = x$, y significa que primero a valía x y que después ha pasado a valer y siguiendo el algoritmo de la poda. Vemos que el resultado es un árbol con menos nodos que el árbol original (atención que en el árbol solución de la pregunta 2 no he pintado todos los nodos!). Es posible que se hubiera podido podar más con otro orden en la generación de los nodos.

Recursos

Para realizar esta PEC el material imprescindible son los temas 2, 3, 4 y 5 del Módulo 2.

Criterios de valoración

La valoración de las preguntas está indicada en el enunciado.

Cada apartado vale el 50% del resultado final.

Formato y fecha de envío

Para dudas y aclaraciones sobre el enunciado, dirigiros al consultor responsable del aula.

Hay que entregar la solución en un archivo **PDF**. Adjuntar el fichero a un mensaje en el apartado Entrega y Registro de EC (REC).

El nombre del archivo debe ser Apellidos_Nombre_IA_PEC1 con la extensión. pdf (PDF).

La fecha límite de entrega es el: **20 de Octubre** (a las 24 horas, más o menos).

Razonad la respuesta en todos los ejercicios. Las respuestas sin justificación no recibirán puntuación.

Nota: Propiedad intelectual

A menudo es inevitable, al producir una obra multimedia, hacer uso de recursos creados por terceras personas. Es por tanto comprensible hacerlo en el marco de una práctica de los estudios de Informática, siempre que se documente claramente y no suponga plagio en la práctica.

Por lo tanto, al presentar una práctica que haga uso de recursos ajenos, se presentará junto con ella un documento en el que se detallen todos ellos, especificando el nombre de cada recurso, su autor, el lugar donde se obtuvo y el su estatus legal: si la obra está protegida por copyright o se acoge a alguna otra licencia de uso (Creative Commons, licencia GNU, GPL ...). El estudiante deberá asegurarse de que la licencia que sea no impide específicamente su uso en el marco de la práctica. En caso de no encontrar la información correspondiente deberá asumir que la obra está protegida por copyright. Deberán, además, adjuntar los archivos originales cuando las obras utilizadas sean digitales, y su código fuente esté corresponde.